AI Revenue Recovery Engine

Recording script & cue sheet — Razorpay Hackathon Track 03

Total time	Structure	Record as
5:00	1:30 setup · 3:30 live · close	2 blocks (see last page)

How to use this sheet: the left column is when and where to look — which window/tab/file to have on screen. The right column is roughly what to say. You don't need to say it word for word — say it your way, hit the same points.

Open before you start recording

- Chrome — tab 1: **docs/architecture-flow.html** (the diagram)
- Chrome — tab 2: Razorpay test dashboard (Payment Links)
- Chrome — tab 3: your dashboard **http://localhost:5173**
- One PowerShell terminal, large font, backend log visible
- **python scripts\dryrun.py** run once — confirm everything is green
- If doing the fully-live version: ngrok running, webhook registered in Razorpay

---

PART A · INTRO + PROBLEM + ARCHITECTURE (0:00 – 1:30)

Time	Screen / file to open	Say (roughly)
0:00 – 0:15	**Your dashboard (home screen)**	“Hi, I'm Shyam. This is an AI revenue recovery engine for Razorpay's Track 3. Everything you'll see today is running live on my laptop — nothing pre-recorded, no hardcoded numbers.”
0:15 – 0:45	**Stay on dashboard**	“When a payment fails, most businesses do one of two things: nothing, or blindly retry it until it works or the customer gives up. Both waste money, because they treat every failure the same. But an expired card and a network glitch are completely different problems. The real question isn't 'retry harder' — it's: given why this specific payment failed, what's the single best next action, and is it even worth taking one?”
0:45 – 1:30	**Tab 1 — architecture-flow.html** ***Sweep top→bottom. Don't read file names. 45 sec max.***	“Here's how it works. A payment fails — either a real Razorpay webhook or a replayed test case. My backend verifies it and hands it to a separate AI agent. That agent can decide, but it cannot touch money — my backend does that, and only after a policy check. Inside the agent: it diagnoses why the payment failed, ranks every possible action by expected value — how likely it is to recover the money versus what it costs — and picks the best one. A policy gate checks it before anything happens. If the amount is over Rs. 25,000, it pauses for a human. Every decision gets written to an audit log, and the system learns from every outcome. That's the whole architecture. Now let me actually run it.”

PART B · LIVE SIMULATION — the core (1:30 – 5:00)

Slow down here. Let each screen sit a second before you talk over it. This is 80% of the video.

Time	Screen / file to open	Say (roughly)
1:30 – 2:15	**Tab 2 — Razorpay test dashboard** ***Open a payment link → pay with test card → click Failure on the simulation screen (or Netbanking → Failure)***	“This is Razorpay test mode — the real API, no real money. I'll fail a payment on purpose... [do it] ...and there it is, a real failed transaction, on Razorpay's own screen. That just fired a payment.failed webhook straight at my backend.”
	Terminal (corner) *Point at the log line the instant it appears*	`LIVE case … opened for failed payment pay_…`

Time	Screen / file to open	Say (roughly)
2:15 – 3:45	**Tab 3 — your dashboard → Cases → the new case → scroll the timeline slowly**	“My system already caught it. This timeline is the real audit trail — not a log I wrote for the demo, this is the actual decision_log table. First, diagnose: it read the failure reason and worked out the underlying cause. Then decide: here's the expected-value ranking it actually saw — every option scored by probability of recovery times amount, minus cost. It picked the top one. Then guard: the policy check, before anything is allowed to happen. Then execute — here's the real action it took.”
	If it RETRIED (transient failure)	“It diagnosed this as a transient glitch, not a real decline, so its best move was an immediate retry. It created a real Razorpay order and is now waiting for the payment to settle.”
	If it sent a LINK (hard decline)	“It diagnosed a hard decline, so retrying does nothing — instead it created a real Razorpay payment link. This URL is live, created through their API.”
	If you see fallback_error	“Here — the AI model failed on this step, so the system fell back to the pure math and still made the correct call, and logged that it fell back. The model ranks options; it isn't the decision-maker. That's why this keeps working even when the model API is down.”

Time	Screen / file to open	Say (roughly)
3:45 – 4:15	**Terminal** `python scripts\webhook_setup.py pay <case-id>`	“Now I'll settle it — this fires the same signed payment-confirmed webhook Razorpay would send. [run it] ...and the case is RECOVERED. Notice: right up until this moment, the action showed pending, not recovered. Creating an order or a link doesn't recover money by itself — only the payment webhook closes the case. This system never counts money it hasn't actually received.”
4:15 – 4:40	**Dashboard → Approvals tab** **Approve the staged high-value case**	“One more thing. Anything above Rs. 25,000 doesn't get auto-handled — it pauses right here and waits for a human. This pause is saved to the database, so even if I restart the whole system, the case is still sitting here. I approve it... and the agent resumes from exactly where it stopped.”

Time	Screen / file to open	Say (roughly)
4:40 – 5:00	**Dashboard — strategy table**	“Across 300 payments: doing nothing recovers zero. Blind retrying gets Rs. 4.65 lakh but hits the gateway 770 times. My system recovers Rs. 15.37 lakh — 3.3 times as much — using less than half the attempts. One honest note: recovery is near the theoretical best because the test set has one right answer per case. The real win is efficiency — this needs far fewer contacts than naive retrying, and it keeps improving as it learns. So: a decision problem, not a retry problem, with a human on the expensive calls and every step on the record. Thank you.”

If the live webhook misbehaves mid-take — don't panic, don't restart services. Run:
`python scripts\webhook_setup.py selftest --reason gateway_technical_error`
Same result — a real case, worked live, case id ready for the pay command. Say: “I'm sending the failure webhook locally so this doesn't depend on the venue wifi” — still fully honest, still shows the same mechanism.

Recording order (not playback order)

1. **Record Part A + the 4:40–5:00 numbers first.** No live state, can't fail — easy wins in the can.
2. **Record 1:30–4:40 as one continuous live take.** If it fumbles, you only re-shoot this block, not the whole video.

Legibility checklist

- Recorder set to 1920×1080 **before** arranging windows
- Chrome zoom 125–150%, terminal font 18+
- Move the mouse slowly; pause 2 seconds after every click
- Record a 20-second throwaway take first and check it back (mic level, text size)

Before you submit

- No secrets visible on screen (.env, terminal scrollback, Razorpay keys)
- Under the time limit
- Link tested in an incognito window